

Modulo 3

Programación en Lenguaje C

- 3.i. Entorno MinGWStudio. Estructura de un programa en C.
- 3.ii. Entrada/Salida en C.
- 3.iii. Estructuras de control.
- 3.iv. Ejercicio de integración.
- 3.v. Vectores.**
- 3.vi. Matrices.**
- 3.vii. Punteros.
- 3.viii. Funciones.
- 3.ix. Ejercicio de integración.

3.v. Vectores, 3.vi. Matrices

■ **Vector:** es una colección de variables del mismo tipo que se referencian por un nombre común. A un elemento específico se accede mediante un índice. También se denominan arrays.

■ **Declaración de un vector unidimensional:** La declaración de un vector unidimensional es:

tipo nombre_de_variable[tamaño];

■ **Tamaño del vector en bytes (8 bits):**

bytes totales = sizeof(tipo base)

*** nº de elementos.**

■ Los arrays tienen **el 0 como índice de su primer elemento**. Por tanto:

char p[tamaño];

// sus elementos son del p[0] al p[9];

```
#include <stdio.h>

int main()
{
    int x[10];
    int t;

    for(t=0;t<10;t++)
        x[t]=t;

    for(t=0;t<10;t++)
        printf("%d",x[t]);

    return 0;
}
```

3.v. Vectores, 3.vi. Matrices

Matrices:

- Las matrices son vectores multidimensionales.
- Su forma más simple son los bidimensionales.

Declaración:

tipo Vector [1ª Dim] [Tam 2ª Dim];

```
int d [5] [8];
```

		Segundo índice							
Primer índice	{	0,0	0,1	0,2	0,3	0,4	0,5	0,6	0,7
		1,0	1,1	1,2	1,3	1,4	1,5	1,6	1,7
		2,0	2,1	2,2	2,3	2,4	2,5	2,6	2,7
		3,0	3,1	3,2	3,3	3,4	3,5	3,6	3,7
		4,0	4,1	4,2	4,3	4,4	4,5	4,6	4,7

```
#include <stdio.h>
int main( )
{
    int t, i, num[3][4];
    for(t=0;t<3;t++)
        for(i=0;i<4;i++)
            num[t][i]=(t*4)+i+1;

    /*Ahora mostrarlos*/
    for(t=0;t<3;t++)
    {
        for(i=0;i<4;i++)
            printf("%d",num[t][i]);
        printf("\n");
    }
    return 0;
}
```

3.v. Vectores, 3.vi. Matrices

■ **Inicialización de vectores:** C permite la inicialización de vectores en el momento de declararlos. La forma general es la siguiente:

Ejemplos:

```
int i[5]={1,2,3,4,5};  
int cuads[4][2]={  
    1,1,  
    2,4,  
    3,9,  
    4,16};
```

■ **Inicialización de vectores indeterminados:** Si no se conoce el número de elementos no es necesario calcularla, se puede omitir.

```
char e1[ ]="vector de caracteres";  
int cuads[ ][2]={  
    1,1,  
    2,4,  
    3,9,  
    4,16};
```

3.v. Vectores, 3.vi. Matrices

```
#include <stdio.h>

int main()
{
    int v[10]={2,4,8,10,12,14,16,18,20,22};
    int i,cantidad;

    printf("Dame la cantidad de n°s\n");
    scanf("%d",&cantidad);
    for(i=0;i<cantidad;i++)
        scanf("%d",&v[i]);
    for(i=0;i<cantidad;i++)
        v[i]=v[i]+i;
    for(i=9;i>-1;i--)
        printf("%d ",v[i]);
    return 0;
}
```

Dame la cantidad de n°s

4

Dame el valor 0

1

Dame el valor 1

2

Dame el valor 2

3

Dame el valor 3

4

22 20 18 16 14 12 7 5 3 1

3.v. Vectores, 3.vi. Matrices

```
#include <stdio.h>
int main()
{
    int i,j,max,min,m[3][2];

    // Leer la matriz
    for(i=0;i<3;i++)
        for(j=0;j<2;j++)
        {
            printf("Dame el valor de la posicion (%d,%d)\n",i,j);
            scanf("%d",&m[i][j]);
        }

    // Mostrar la matriz por pantalla
    printf("MATRIZ\n");
    for(i=0;i<3;i++)
    {
        printf("\t");
        for(j=0;j<2;j++)
            printf("%d ",m[i][j]);
        printf("\n");
    }
    printf("\n");

    // Maximo por filas
    for(i=0;i<3;i++)
    {
        max=m[i][0];
        for(j=0;j<2;j++)
        {
            if (max<m[i][j]) max=m[i][j];
        }
        printf("El maximo de la fila %d es %d\n",i,max);
    }

    // Minimo por columnas
    for(i=0;i<2;i++)
    {
        min=m[0][i];
        for(j=0;j<3;j++)
        {
            if (min>m[j][i]) min=m[j][i];
        }
        printf("El minimo columna %d es %d\n",i,min);
    }

    return 0;
}
```

Dame el valor de la posicion (0,0)

3

Dame el valor de la posicion (0,1)

4

Dame el valor de la posicion (1,0)

1

Dame el valor de la posicion (1,1)

-2

Dame el valor de la posicion (2,0)

7

Dame el valor de la posicion (2,1)

6

MATRIZ

3	4
1	-2
7	6

El maximo de la fila 0 es 4

El maximo de la fila 1 es 1

El maximo de la fila 2 es 7

El minimo columna 0 es 1

El minimo columna 1 es -2

3.v. Vectores, 3.vi. Matrices

■ **Cadenas de caracteres:** Es un vector de caracteres de cualquier longitud que termina en un carácter nulo.

■ El **nulo se especifica como '\0'**, por esta razón es necesario que los vectores de caracteres sean de un carácter más que la cadena más larga que pueda contener. **Ejemplo:**

```
char s[11]; /*cadena que puede contener 10 caracteres*/
```

■ Funciones de manejos de cadenas, las más comunes son:

```
char *strcpy(char *s1,const char *s2);
```

```
char *strcat(char *s1,const char *s2);
```

```
int strlen(const char *s2);
```

```
int strcmp(const char *s1,const char *s2);
```

■ La función **strcpy()** copia la cadena apuntada por s2 en la cadena apuntada por s1.

■ La función **strcat()** concatena la cadena apuntada por s2 en la apuntada por s1.

■ La función **strlen()** devuelve la longitud de la cadena apuntada por s1.

■ La función **strcmp()** compara dos cadenas, devuelve 0 si son iguales, mayor que 0 si la cadena apuntada por s1 es mayor que la puntada por s2, y menor que 0 si s2 es mayor que s1.

3.v. Vectores, 3.vi. Matrices

Ejemplo: Programa que lee dos cadenas de caracteres, muestra su longitud por pantalla, dice si son iguales, las concatena y muestra por pantalla.

```
#include <stdio.h>
#include <string.h>
int main( )
{
    char c1[80],c2[80];

    gets(c1);
    gets(c2);
    printf("longitudes: %d,%d\n", strlen(c1),
          strlen(c2));

    if(!strcmp(c1,c2)) printf("Las cadenas son
                             iguales\n");
    strcat(c1,c2);
    printf("%s\n",c1);

    return 0;
}
```

```
cadena1
cadena2
longitudes: 7,7
cadena1cadena2
```

```
cadena
cadena
longitudes: 6,6
Las cadenas son iguales
cadenacadena
```